INF201 - Lecture 8

26th Oct 2026

Topics

- Organising your code
- Object-oriented programming
  - Introduction
  - Creating a class

Organising your code

When you work on projects, it is a good idea to separate between:

1. Reusable code - in Python scripts
2. Interactive analysis or visualisation - in Jupyter Notebooks
3. Documentation - in README.md and Jupyter Notebooks

Why work like this?

- Makes it easier to re-use parts of the code for this or other projects
  - We can for example import a function from another python file by `from my_analysis_code import summary_statistics`
- Cleaner git diffs
  - One-liners in `.py` files
  - Includes all metadata in `.ipynb` files
  - Consider using the nbstripout package to "clean up" your notebooks before committing them to git
- Here are a couple of excellent resources on the topic:
  - The Good Research Handbook
  - Good enough practices in scientific computing

Your project could be organised something like this:

```
    project_name/
    ├── .gitignore
    ├── README.md
    ├── requirements.txt           # info about your venv
    ├── data/                      # consider adding this folder to
.gitignore
    │   ├── raw/
    │   └── processed/
    ├── notebooks/                 # your jupyter notebook(s)
    │   ├── 01_data_cleaning.ipynb
```

```
    │   └── 02_model_evaluation.ipynb
    ├── src/                        # python scripts with the majority
of your code
    │   ├── data.py                 # functions for loading/transforming
data
    │   └── processing.py           # functions, classes, processing
data, ...
    └── tests/                      # folder for testing (more about
this later)
        └── test_processing.py
```

- An `src` folder typically contains the resusable logic for the project
  - functions
  - classes
  - data pipelines
- A `notebooks` or `scripts` folder contains single purpose scripts/notebooks
  - These will import code from `src`
  - The scripts/notebooks here are run directly, unlike scripts in `src`

A recap: How to write good code

- Style
  - Follow PEP-8
  - This includes sensible names for variables, functions and classes
- Functions
  - If you do something more than once, it should be a function!
  - Many small functions are better than few large functions
  - I recommend writing all functions at the beginning of your scripts
  - Use doc-strings and type hints
  - A script with several functions will typically have a `main()` function that runs when the script itself runs,
    but not when it is imported to other scripts
    ```python
    if __name__ == "__main__":
        main()
    ```
- Efficency
  - Chose appropriate data containers
  - Vectorise instead of using loops if possible
- Structure and reproducibility
  - Use virtual environments
  - Organise your data, code and notebooks
  - Use version control (like Git)

Let's make a game!

- We want to create an RPG in Python
  - The players are characters on an adventure
  - They get into battles

    - They find items
- The script `game.simulator_v0.py` contains a couple of useful functions, let's have a look
    - In Jypter notebook, you can use `?` to get information on a function or `??` to show the code itself as well

In [46]:
```
import game_simulator_v0 as g0

g0.take_damage?
```

```
Signature: g0.take_damage(player: dict, amount: int) -> None
Docstring: Player hp is updated after taking amount damage
File:      ~\documents\inf201-h26-my-repo\lectures\lecture_08\game_simulator_v0.p
y
Type:      function
```

In [47]:
```
g0.take_damage??
```

```
Signature: g0.take_damage(player: dict, amount: int) -> None
Source:
def take_damage(player: dict, amount: int) -> None:
    """Player hp is updated after taking amount damage"""
    player["hp"] -= amount
    if player["hp"] < 0:
        player["hp"] = 0
File:      ~\documents\inf201-h26-my-repo\lectures\lecture_08\game_simulator_v0.p
y
Type:      function
```

In [48]:
```
g0.heal??
```

```
Signature: g0.heal(player: dict, amount: int) -> None
Source:
def heal(player: dict, amount: int) -> None:
    """Player hp is updated after healing amount points"""
    player["hp"] += amount
File:      ~\documents\inf201-h26-my-repo\lectures\lecture_08\game_simulator_v0.p
y
Type:      function
```

In [49]:
```
# Introduce a player
player1 = {"name": "Aria", "hp": 100, "inventory": ["potion", "shield"]}

# The player can be injured
g0.take_damage(player1, 10)
print(f"{player1["name"]} is injured and now has {player1["hp"]} hit points")

# The player can be healed
g0.heal(player1, 5)
print(f"{player1["name"]} heals and now has {player1["hp"]} hit points")
```

```
Aria is injured and now has 90 hit points
Aria heals and now has 95 hit points
```

- What happens if we have several players?
- How do we make sure that HP stays within allowed boundaries?
- How do we keep track of which functions belong to which entities?
    - Which actions should players be able to do?

- What if different players have different available actions?

In [50]:
```
# Nothing is preventing us from setting HP to "illegal" values
player1["hp"] = -999
player1
```

Out[50]:
```
{'name': 'Aria', 'hp': -999, 'inventory': ['potion', 'shield']}
```

- We will look at an alternative approach for this task

Object-oriented programming

- The first object-oriented programming language was developed in Norway!
  - Simula
  - Created by Ole-Johan Dahl and Kristen Nygaard in the 1960s
- Mini intro to object-oriented Python
- Object-orented programming - Wikipedia article
- YouTube has some excellent tutorials on the topic
- This is one of several programming paradigmes:
  - Procedural: Procedures (e.g. functions) calling each other
  - Object-oriented: Objects containing both data and associated methods

Aspect	Procedural Approach	Object-Oriented Approach
Core Focus	Functions acting on standalone data (`fn(data)`)	Objects bundling data and behavior (`object.method()`)
Data Organization	Scattered variables and containers (`player["hp"]`)	Structured attributes defined in a custom class (`player.hp`)
State Protection	Data is exposed: any function can mutate it accidentally	Encapsulated: state is protected and updated via explicit methods
Code Structure	Sequential step-by-step logic and utility scripts	Modular, self-contained components with clear public interfaces
IDE Ergonomics	Limited autocomplete: prone to typos in dictionary keys	Full VS Code IntelliSense, real-time type checking, and inline docs
Best Used For	Quick scripts, data pipelines, and linear data transformations	Large codebases, complex state management, and team collaboration

> Python is an object-oriented programming language, and in Python everything is an object.

The Whirlwind Tour of Python by Jake VanderPlas

- Good news: You already use object-oriented programming
- We look at `numpy` as an example

In [51]:
```
import numpy as np
```

```
ones_array = np.ones((2, 3), dtype=np.int8)
ones_array
```

Out[51]:
```
array([[1, 1, 1],
       [1, 1, 1]], dtype=int8)
```

In [52]:
```
np.ones?
```

```
Signature: np.ones(shape, dtype=None, order='C', *, device=None, like=None)
Docstring:
Return a new array of given shape and type, filled with ones.

Parameters
----------
shape : int or sequence of ints
    Shape of the new array, e.g., ``(2, 3)`` or ``2``.
dtype : data-type, optional
    The desired data-type for the array, e.g., `numpy.int8`.  Default is
    `numpy.float64`.
order : {'C', 'F'}, optional, default: C
    Whether to store multi-dimensional data in row-major
    (C-style) or column-major (Fortran-style) order in
    memory.
device : str, optional
    The device on which to place the created array. Default: None.
    For Array-API interoperability only, so must be ``"cpu"`` if passed.

    .. versionadded:: 2.0.0
like : array_like, optional
        Reference object to allow the creation of arrays which are not
        NumPy arrays. If an array-like passed in as ``like`` supports
        the ``__array_function__`` protocol, the result will be defined
        by it. In this case, it ensures the creation of an array object
        compatible with that passed in via this argument.

    .. versionadded:: 1.20.0

Returns
-------
out : ndarray
    Array of ones with the given shape, dtype, and order.

See Also
--------
ones_like : Return an array of ones with shape and type of input.
empty : Return a new uninitialized array.
zeros : Return a new array setting values to zero.
full : Return a new array of given shape filled with value.

Examples
--------
>>> import numpy as np
>>> np.ones(5)
array([1., 1., 1., 1., 1.])

>>> np.ones((5,), dtype=np.int_)
array([1, 1, 1, 1, 1])

>>> np.ones((2, 1))
array([[1.],
       [1.]])

>>> s = (2,2)
>>> np.ones(s)
array([[1.,  1.],
       [1.,  1.]])
File:      ~\documents\inf201-h26-my-repo\.venv\lib\site-packages\numpy\_core\num
```

```
eric.py
Type:      function
```

In [53]:
```
print(f"Is ones_array an object? {isinstance(ones_array, object)}")
```

```
Is ones_array an object? True
```

In [54]:
```
print(f"Type: {type(ones_array)}")
```

```
Type: <class 'numpy.ndarray'>
```

In [55]:
```
print(f"Data type attribute: {ones_array.dtype}")
print(f"Shape attribute: {ones_array.shape}")
```

```
Data type attribute: int8
Shape attribute: (2, 3)
```

In [56]:
```
print("Methods:")
print(dir(ones_array))
```

```
Methods:
['T', '__abs__', '__add__', '__and__', '__array__', '__array_finalize__', '__arra
y_function__', '__array_interface__', '__array_namespace__', '__array_priority_
_', '__array_struct__', '__array_ufunc__', '__array_wrap__', '__bool__', '__buffe
r__', '__class__', '__class_getitem__', '__complex__', '__contains__', '__copy_
_', '__deepcopy__', '__delattr__', '__delitem__', '__dir__', '__divmod__', '__dlp
ack__', '__dlpack_device__', '__doc__', '__eq__', '__float__', '__floordiv__', '_
_format__', '__ge__', '__getattribute__', '__getitem__', '__getstate__', '__gt_
_', '__hash__', '__iadd__', '__iand__', '__ifloordiv__', '__ilshift__', '__imatmu
l__', '__imod__', '__imul__', '__index__', '__init__', '__init_subclass__', '__in
t__', '__invert__', '__ior__', '__ipow__', '__irshift__', '__isub__', '__iter__',
'__itruediv__', '__ixor__', '__le__', '__len__', '__lshift__', '__lt__', '__matmu
l__', '__mod__', '__mul__', '__ne__', '__neg__', '__new__', '__or__', '__pos__',
'__pow__', '__radd__', '__rand__', '__rdivmod__', '__reduce__', '__reduce_ex__',
'__repr__', '__rfloordiv__', '__rlshift__', '__rmatmul__', '__rmod__', '__rmul_
_', '__ror__', '__rpow__', '__rrshift__', '__rshift__', '__rsub__', '__rtruediv_
_', '__rxor__', '__setattr__', '__setitem__', '__setstate__', '__sizeof__', '__st
r__', '__sub__', '__subclasshook__', '__truediv__', '__xor__', '_set_dtype', '_se
t_shape', 'all', 'any', 'argmax', 'argmin', 'argpartition', 'argsort', 'astype',
'base', 'byteswap', 'choose', 'clip', 'compress', 'conj', 'conjugate', 'copy', 'c
types', 'cumprod', 'cumsum', 'data', 'device', 'diagonal', 'dot', 'dtype', 'dum
p', 'dumps', 'fill', 'flags', 'flat', 'flatten', 'getfield', 'imag', 'item', 'ite
msize', 'mT', 'max', 'mean', 'min', 'nbytes', 'ndim', 'nonzero', 'partition', 'pr
od', 'put', 'ravel', 'real', 'repeat', 'reshape', 'resize', 'round', 'searchsorte
d', 'setfield', 'setflags', 'shape', 'size', 'sort', 'squeeze', 'std', 'strides',
'sum', 'swapaxes', 'take', 'to_device', 'tobytes', 'tofile', 'tolist', 'trace',
'transpose', 'var', 'view']
```

In [58]:
```
# ones_array.sum()
ones_array.sum(dtype=np.int8)
```

Out[58]:
```
np.int8(6)
```

In [59]:
```
print(f"Is np.sum() an object? {isinstance(np.sum, object)}")
print(f"Type: {type(np.sum)}")
```

```
Is np.sum() an object? True
Type: <class 'numpy._ArrayFunctionDispatcher'>
```

In [60]:
```
isinstance(np, object)
```

Out[60]: True

Object-oriented concepts

- **Class**: A template for creating an object
- **Object**: An instance of a class
- **Attribute**: Variables that belong to a class
- **Method**: Function that operates on objects of the class

Creating a Class

- Created by `class` in Python
- Similar to creating functions
- Should also use doc-strings and type hints
- Class names use `CamelCase`
- `example_module.py` has a PEP8 compliant template for writing classes

In [61]:
```
from example_module import ClassName

obj = ClassName(param1="Hello", param2="World")
print(obj.method_name("Test"))
print(ClassName.utility_method("Check"))
default_obj = ClassName.create_default()
print(default_obj.method_name("Default Test"))
```

```
Processed Test with Hello and World
True
Processed Default Test with default1 and default2
```

- Let's make a character class for our RPG game
  - We want the character to be able to fight
- Which attributes does the character need?
  - Name
  - Health (hp), both current and max/min
  - Damage, i.e. how much damage does the character deal when attacking?
- What should the character be able to do (i.e. methods)?
  - Attack

1. Set up the class with a name and a doc-string

In [ ]:
```
class Character:
    """Represents a character in the RPG"""
```

2. Initialise using `__init__` and `self`
   - The `__init__` method is the *constructor* for this class
   - It is used to create a specific object of this class
   - `self` is a placeholder saying "belonging to whichever specific object is currently running this code"

- We use type hints for the attributes

In [62]:
```python
class Character:
    """Represents a character in the RPG"""

    def __init__(self, name: str, hp_max: int = 100, damage: int = 10) -> None:
        """
        Initialise the Character

        Args:
            name (str): The name of the character.
            hp_max (int): Max hit points (default = 100)
            damage (int): Damage delt when character attacks (default = 10)
        """
        self.name = name
        self.hp = hp_max  # Assume the character is created with max hp
        self.hp_max = hp_max
        self.damage = damage
```

3. Add a method that allows the character to attack
    - This method is linked to `self` and a differnt object (here called `target`)

In [65]:
```python
class Character:
    """Represents a character in the RPG"""

    def __init__(self, name: str, hp_max: int = 100, damage: int = 10) -> None:
        """
        Initialise the Character

        Args:
            name (str): The name of the character.
            hp_max (int): Max hit points (default = 100)
            damage (int): Damage delt when character attacks (default = 10)
        """
        self.name = name
        self.hp = hp_max  # Assume the character is created with max hp
        self.hp_max = hp_max
        self.damage = damage

    def attack(self, target: Character) -> None:
        """Attack and deal damage to target Character"""
        target.hp -= self.damage
        target.hp = max(target.hp, 0)  # Target hp cannot end up lower than 0
```

We can now create two characters and have them attack each other

In [66]:
```python
character1 = Character(name="Aria", hp_max=100, damage=10)
character2 = Character(name="Kura", hp_max=100, damage=20)

print(f"{character1.name} has {character1.hp} out of {character1.hp_max} left")
print(f"{character2.name} has {character2.hp} out of {character2.hp_max} left")
```

```
Aria has 100 out of 100 left
Kura has 100 out of 100 left
```

In [67]:
```python
character1.attack(character2)
print(f"{character1.name} has {character1.hp} out of {character1.hp_max} left")
```

```
print(f"{character2.name} has {character2.hp} out of {character2.hp_max} left")
```

```
Aria has 100 out of 100 left
Kura has 90 out of 100 left
```

In [68]:
```
character2.attack(character1)
print(f"{character1.name} has {character1.hp} out of {character1.hp_max} left")
print(f"{character2.name} has {character2.hp} out of {character2.hp_max} left")
```

```
Aria has 80 out of 100 left
Kura has 90 out of 100 left
```

- Let's talk about `self` again here
- Python reads `character1.attack(character2)` as `Character.attack(character1, character2)`
- The dot notation is *syntactic sugar* where the instance to the left is automatically passed as the first argument of the function
- If you forget `self` in the method definition, you will run into trouble

In [71]:
```
class Character:
    """Represents a character in the RPG"""

    def __init__(self, name: str, hp_max: int = 100, damage: int = 10) -> None:
        """
        Initialise the Character

        Args:
            name (str): The name of the character.
            hp_max (int): Max hit points (default = 100)
            damage (int): Damage delt when character attacks (default = 10)
        """
        self.name = name
        self.hp = hp_max  # ASsume the character is created with max hp
        self.hp_max = hp_max
        self.damage = damage

    def attack(self, target) -> None:
        """Attack and deal damage to target character"""
        target.hp -= self.damage
        target.hp = max(target.hp, 0)  # Target hp cannot end up lower than 0

    def introduce() -> None:  # Here we forgot to include self!
        """Creates string that introduces character"""
        print(f"Hi, my name is {self.name}")
```

In [72]:
```
character1 = Character(name="Aria", hp_max=100, damage=10)
character1.introduce()
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[72], line 2
      1 character1 = Character(name="Aria", hp_max=100, damage=10)
----> 2 character1.introduce()

TypeError: Character.introduce() takes 0 positional arguments but 1 was given
```

In [73]:
```
Character.introduce(character1)
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[73], line 1
----> 1 Character.introduce(character1)

TypeError: Character.introduce() takes 0 positional arguments but 1 was given
```

In [74]:
```python
class Character:
    """Represents a character in the RPG"""

    def __init__(self, name: str, hp_max: int = 100, damage: int = 10) -> None:
        """
        Initialise the Character

        Args:
            name (str): The name of the character.
            hp_max (int): Max hit points (default = 100)
            damage (int): Damage delt when character attacks (default = 10)
        """
        self.name = name
        self.hp = hp_max  # Assume the character is created with max hp
        self.hp_max = hp_max
        self.damage = damage

    def attack(self, target) -> None:
        """Attack and deal damage to target character"""
        target.hp -= self.damage
        target.hp = max(target.hp, 0)  # Target hp cannot end up lower than 0

    def introduce(self) -> None:
        """Creates string that introduces character"""
        print(f"Hi, my name is {self.name}!")
```

In [75]:
```python
character1 = Character(name="Aria", hp_max=100, damage=10)
character1.introduce()
```

```
Hi, my name is Aria!
```

4. Make objects inspectable using `__repr__` and `__str__`
    - `__repr__`: For developers and debuggers. Should ideally look like valid Python code to recreate the object, or a unambiguous string.
    - `__str__`: For end users and clean outputs (`print()` statement)

In [76]:
```python
character1  # will call __repr__
```

Out[76]:
```
<__main__.Character at 0x14ec9f302f0>
```

In [77]:
```python
print(character1)  # will call __str__
```

```
<__main__.Character object at 0x0000014EC9F302F0>
```

In [78]:
```python
ones_array  # __repr__ with more useful output
```

Out[78]:
```
array([[1, 1, 1],
       [1, 1, 1]], dtype=int8)
```

In [79]:
```python
print(ones_array)  # __str__ with more useful output
```

```
[[1 1 1]
 [1 1 1]]
```

In [82]:
```python
class Character:
    """Represents a character in the RPG"""

    def __init__(self, name: str, hp_max: int = 100, damage: int = 10) -> None:
        """
        Initialise the Character

        Args:
            name (str): The name of the character.
            hp_max (int): Max hit points (default = 100)
            damage (int): Damage delt when character attacks (default = 10)
        """
        self.name = name
        self.hp = hp_max  # Note: _hp indicated private attribute
        self.hp_max = hp_max
        self.damage = damage

    def attack(self, target) -> None:
        """Attack and deal damage to target character"""
        target.hp -= self.damage
        target.hp = max(target.hp, 0)  # Target hp cannot end up lower than 0

    def introduce(self) -> None:
        """Creates string that introduces character"""
        print(f"Hi, my name is {self.name}!")

    def __repr__(self) -> str:
        """Representation of character object"""
        return f"<Character name={self.name} hp={self.hp} damage={self.damage}>)

    def __str__(self) -> str:
        """String representation of character object"""
        return f"Character: {self.name} with {self.hp} HP and {self.damage} dama
```

In [83]:
```python
character1 = Character(name="Aria", hp_max=100, damage=10)
character1  # will call __repr__()
```

Out[83]:
```
<Character name=Aria hp=100 damage=10>)
```

In [84]:
```python
print(character1)  # will call __str__()
```

```
Character: Aria with 100 HP and 10 damage
```

5. Protect internal state (encapsulation using `@property`)
    - We want to prevent external code to setting class attributes to invalid values
        - Example: Keep `hp` between `0` and `hp_max`
    - For this we start the attribute name with a single underscore `_`
    - This signals that this attribute should be considered protected (internal data), meaning it should only be accessed within the class (or by subclasses)
    - This is a naming convention, and will not be enforced
    - By also including `@property`, we can enforce this

In [85]:
```python
class Character:
    """Represents a character in the RPG"""

    def __init__(self, name: str, hp_max: int = 100, damage: int = 10) -> None:
        """
        Initialise the Character

        Args:
            name (str): The name of the character.
            hp_max (int): Max hit points (default = 100)
            damage (int): Damage delt when character attacks (default = 10)
        """
        self.name = name
        self._hp = hp_max  # Note: _hp indicated private attribute
        self.hp_max = hp_max
        self.damage = damage

    def attack(self, target) -> None:
        """Attack and deal damage to target character"""
        target._hp -= self.damage
        target._hp = max(target._hp, 0)  # Target hp cannot end up lower than 0

    def introduce(self) -> None:
        """Creates string that introduces character"""
        print(f"Hi, my name is {self.name}!")

    def __repr__(self) -> str:
        """Representation of character object"""
        return f"<Character name={self.name} hp={self.hp} damage={self.damage}>)

    def __str__(self) -> str:
        """String representation of character object"""
        return f"Character: {self.name} with {self.hp} HP and {self.damage} dama

    @property
    def hp(self) -> int:
        """Public read-only property to inspect HP without allowing direct, unsa
        return self._hp
```

In [86]:
```python
character1 = Character(name="Aria", hp_max=100, damage=10)
character1.hp
```

Out[86]: 100

In [87]:
```python
character1.hp = 50
```

```
---------------------------------------------------------------------------
AttributeError                            Traceback (most recent call last)
Cell In[87], line 1
----> 1 character1.hp = 50

AttributeError: property 'hp' of 'Character' object has no setter
```

In [88]:
```python
character1._hp
```

Out[88]: 100

In [89]:
```python
character1._hp = 50
```

```
In [90]: character1._hp
```

```
Out[90]: 50
```

- `_hp` is the private container holding the raw value
- `@property` is the gatekeeper controlling how the outside world views or changes that value.

What does @ mean here?

- `@` methods in Python are called decorators
- A decorator is syntactic sugar that wraps a function or method to modify or extend its behavior without altering its underlying code
- `@property` turns a method into an attribute with hidden logic
- In our example `@property` creates an attribute `hp` that can be viewed but not changed (until we define a method for it)
- We can therefore view it by `character1.hp`, but not change it
- However, we can still change the attribute `_hp`
- Think of `hp` as the front door where the general audience will view and access the attribute
  while `_hp` is the backdoor where developer can get access

What's the deal with `__double_underscore__` methods?

- Methods using double underscores are sometimes referred to as *special methods*, dunder methods or magic methods
- What are dunder methods?
- Pyhton documentation: The data model is a good reference for all these methods
- These methods indicate how the class interacts with built-in Python methods and operators
- Some examples:
  - `__init__`: Creating an object
  - `__repr__` and `__str__`: Controls how an object is printed and displayed
  - `__add__`: Defines how objects are added, how `+` works
  - `__len__`: Defines how the length of the object is determined

Summary of our Character class so far

- We have create a class with an initialiser
- Objects of this class has attributes
  - `name`
  - `_hp`
  - `hp_max`
  - `damage`
- And a property attribute

    - `hp` - protecting `_hp`
- Objects of this class have some regular methods
    - `attack()` to attack another object
    - `introduce()` to introduce themselves
- ... and some special methods
    - `__repr__()` for representation
    - `__str__()` for display representation

We might also want

- Sub-classes for different types of characters
- Methods representing other actions
- Methods for generating default characters
- Different classes, for example Dice
- Inventories with actions depending on inventory items